RepSense: Project Guide

> **What this is:** a guide to how RepSense works.
>
> - **Part 1** explains the machine-learning ideas.
> - **Part 2** covers the AI (Gemini) features.
> - **Part 3** walks through the code, file by file.
> - **Part 4** traces what happens during a workout, from camera frame to saved stats.
>
> Written on 12 Nov 2026, after Phase 2 Step 1 (46 exercises).

The big picture

RepSense watches you exercise through your phone camera and coaches you. **Everything that looks at your body runs inside your browser.** Video never leaves the device; only numbers (reps, scores, times) are saved to the database.

```
Camera frame
    │
    ▼
(1) Pose estimation (MediaPipe)  ── 33 body points per frame
    │
    ├──► (2) Rule engine (our code) ── counts reps, times holds, checks form, speaks cues
    │
    └──► (3) Exercise classifier (our trained model) ── "you're doing squats" (Auto mode)
    │
    ▼
Workout stats ──► Supabase (database), only numbers
```

There are three kinds of "intelligence" in the app. They're worth keeping apart in your head:

	What it is	Who made it	Where it runs
① **Pose estimation**	A pretrained neural network that finds body joints in an image	Google (MediaPipe)	Your browser
② **Rule engine**	Geometry + signal processing + state machines: rep counting and form checks	Us (`web/lib/exercises`, `web/lib/workout`)	Your browser
③ **Exercise classifier**	A 1D-CNN that recognises *which* exercise you're doing	**Us**: trained in `ml/`, exported to ONNX	Your browser
④ **Gemini (LLM)**	Reads meal descriptions and photos, writes diet plans, chats as a coach	Google, called through our server	Our server → Google

① and ② do most of the work during a workout. ③ is the part *we* trained, and the main ML piece of the project. ④ is a separate feature (the Diet and Coach tabs).

Part 1: The machine learning, step by step

1.1 Pose estimation: from pixels to body points

Model: MediaPipe Pose Landmarker (lite), file `web/public/models/pose_landmarker_lite.task`.

For every camera frame it returns **33 landmarks** (nose, shoulders, elbows, wrists, hips, knees, ankles, heels, toes…). Each landmark has:

Field	Meaning
`x`, `y`	Position in the image, **normalised 0–1** (0,0 = top-left). `y` grows **downwards**.
`z`	Rough depth (not used much)
`visibility`	0–1: how sure the model is that the point is visible. We ignore points below **0.5** (`MIN_VISIBILITY`).

- **We didn't train this model.** We use it as a fixed "sensor".
- **It runs on the GPU when it can** and falls back to the CPU (`web/lib/pose/landmarker.ts`).
- **Hidden joints are still guessed.** From a side view the far arm is hidden, but MediaPipe still guesses where it is. That's why `visibility` matters.

1.2 Cleaning the signal

Raw landmarks jitter from frame to frame. Four small steps make them usable:

Step	File	Idea
Smoothing	`web/lib/pose/filters.ts`	**One-Euro filter** (Casiez et al., 2012): a low-pass filter whose cutoff rises with speed. When you hold still it smooths heavily, so there's no jitter; when you move fast it barely smooths, so there's no lag. See `docs/one-euro.png`.
Which side?	`web/lib/pose/side.ts`	From the side, one arm and one leg are hidden. Measure on the side with better visibility, and only switch when the other side is clearly better (margin 0.15), so it doesn't flicker.
Which view?	`web/lib/pose/view.ts`	Shoulder width ÷ torso length: > 0.45 means front view, < 0.30 means side view, in between keeps the last answer. Some form rules only make sense from one view.
In frame?	`web/lib/pose/frame-check.ts`	Enough of the body visible (head, shoulders/hips, a foot) to track at all?

The core maths is in `web/lib/pose/geometry.ts`. The angle at joint B, formed by points A–B–C:

```
θ = arccos( (BA · BC) / (|BA| · |BC|) )
```

For example, `angle(hip, knee, ankle)` is the knee angle: about 175° standing and about 90° at a parallel squat.

> **Pixels, not normalised units:** angles are computed in **pixels**. On a 16:9 frame, 0.1 in x is a longer distance than 0.1 in y, so measuring in normalised units would bend every angle.

1.3 The rule engine: counting reps without training data

This part isn't machine learning: it's **signal processing + a state machine**. It's what counts your reps today, for all 46 exercises.

Signals: one number per exercise

Every rep exercise boils down to **one number that drops and comes back**. A *signal* (`web/lib/exercises/signals.ts`) turns a pose into that number:

Signal	Measures	Used for
`knee`	hip–knee–ankle angle	squats
`front-knee`	the more bent knee	lunges, pistol squat
`hip`	shoulder–hip–knee angle	deadlifts, bridges, swings
`elbow`	shoulder–elbow–wrist angle	push-ups, pull-ups, rows, curls, presses
`arm-raise`	180° − hip–shoulder–elbow angle	lateral and front raises (**flipped**, because the arm angle *grows* during a raise)
`heel-raise`	90° − foot tilt	calf raises (flipped)
`body-line`	how far shoulder–hip–ankle bends from a straight line	plank, side plank

The rep counter: a state machine with hysteresis

`web/lib/exercises/state-machine.ts`, `createRepCounter`:



- **Two thresholds, not one (hysteresis).** For a squat you go "down" below 100° and the rep counts back above 160°. Jitter around a single line can't count twice, because you have to cross *both* lines.
- **Rejected attempts** are reported, so the app can coach you:
- `shallow`: didn't go deep enough ("Go lower");
- `no-lockout`: went back down before reaching the top;
- `too-fast`: faster than `minRepMs`, usually a tracking glitch. The negative pull-up uses this on purpose, needing at least 2.5 s.
- `goal`: an extra condition failed. For pull-ups that's "chin over the bar", approximated as the nose above the wrists.

- **Holds** (plank, wall sit…) use `createHoldTimer`: a timer that runs only while you're in position *and* within tolerance, with a 600 ms grace period for tracking blips.
- For "band" holds such as a wall sit, the measurement is **how many degrees outside the target range** you are (knee between 75° and 110°).

Form checks and scoring

- **Form rules** (`web/lib/exercises/rules.ts`) are small pure functions. Each returns the joints to highlight if broken, or `null` if fine (or not measurable). Examples:
- knees caving (front view only)
- hips sagging or piking
- torso leaning
- knee past toes
- hinge at the hips
- don't swing
- stop at shoulder height
- elbows flaring
- `web/lib/workout/form.ts` turns the checks into coaching:
- an issue must persist **300 ms** before it counts (one noisy frame shouldn't mark you as bad);
- each rep starts at 100 points and loses **35 per "bad" issue** and **15 per "warn" issue**;
- a rep scoring **≥ 70** is a "good rep";
- spoken cues are rate-limited so the app doesn't nag.

Families and the catalog (Phase 2 Step 1)

- **The catalog is data.** Each exercise is a few lines in `web/lib/exercises/catalog.ts`: family, thresholds, rules, equipment.
- **Families supply the defaults** that exercises share (`families.ts`): which signal, camera view, setup hint, cues, MET.
- `build.ts` **merges the two** into the config the engine runs.
- **Adding an exercise means adding data, not writing code.**

> **Honest note:** the thresholds for the original 6 exercises were tuned and are guarded by regression tests. The 40 added in Step 1 are **estimates from typical joint ranges** and haven't been tested on real people yet (Step 4).

1.4 The exercise classifier: the part we trained

Goal: in "Auto" mode, recognise *which* exercise you're doing, so you don't have to pick it. **Classes (7):** `idle, squat, pushup, pullup, dip, lunge, plank`. Idle is a "negative class": standing, walking, resting.

The pipeline lives in `ml/` (Python) and mirrors exactly into the browser (TypeScript):

```
Recorded clips (JSON landmarks)          ml/data/
       │  python -m src.prepare
       ▼
Windows of features (.npz)               ml/data/processed/
       │  python -m src.train
       ▼
Random Forest + 1D-CNN, evaluated        ml/reports/RESULTS*.md, ml/models/
       │  python -m src.export
       ▼
exercise_classifier.onnx + .json         web/public/models/
       │  (browser) ONNX Runtime Web
       ▼
"squat, 92 % sure" → Auto switches the rep counter
```

Step 1: data

- **Recorded in the app.** Clips come from the in-app recorder (**Record** screen, then **Clips**, then **Export all**) as JSON: one row of 33 landmarks per frame, plus the label, view and **subject** (who recorded it). The format is documented in `ml/README.md`.
- **Or converted from video:** `ml/src/extract_landmarks.py` turns `.mp4` files into the same format, using the same pose model.
- **Synthetic stick figures:** `ml/src/synthetic.py` generates figures "performing" each exercise. They're only for testing the pipeline: **they're far cleaner than real people, so nothing measured on them counts as a result.**

Step 2: features (34 numbers per frame)

`ml/src/features.py`, with a TypeScript twin in `web/lib/ml/features.ts`: - **13 joints × (x, y) = 26 numbers:** - converted to pixels; - **centred on the hips**, so where you stand in the frame doesn't matter; - **scaled by body size**, the larger of torso length and shoulder width, so your distance from the camera doesn't matter; - clipped to ±4. - **8 joint angles** (both elbows, shoulders, hips and knees), scaled to 0–1.

> **Why hand-made features instead of raw pixels?** We have very little data. Giving the model body-relative coordinates plus angles bakes in knowledge it would otherwise need thousands of clips to learn.

Step 3: windows (a short video of numbers)

- **Resampled to 15 Hz.** The 15 or 30 FPS from the phone is converted by *time*, not frame count, so fast and slow phones look the same.
- **Window = 30 frames = 2 seconds**, long enough to contain most of a rep. Windows start every 5 frames (stride).
- **Tracking gaps:** if the person was lost for more than 250 ms, windows never cross the gap.
- **Shape of one training example:** `30 × 34`.

Step 4: splitting (the most important ML idea here)

`ml/src/split.py` uses **GroupKFold by person**: every window from one person goes into the *same* fold. - **Why not a random split?** Neighbouring windows overlap almost completely. A random split would test the model on near-copies of its training data, and give a fake ~100 %. - **The question we actually care about:** *does it work on someone it has never seen?* Only a person-wise split answers it. - **Fallback:** with fewer than 3 people it falls back to splitting by clip, and says so in the report.

Step 5: augmentation (training folds only)

- **Variants of each window:**
- **Mirror:** swap left and right, flip x, like a left-handed person doing the same exercise.
- **Jitter:** a small rotation (±10°), scale (±10 %) and per-frame noise.
- **Result:** originals + mirrored + jittered copies of both = **4× the data**.
- **Only training folds are augmented**, never test folds, otherwise test information leaks into training.

Step 6: two models, compared fairly

	Random Forest (baseline)	**1D-CNN (shipped)**
Input	Mean, std, min and max of each feature over the window (**loses time order**)	The full 30 × 34 window
Idea	Many decision trees vote	Small filters slide **along time** and learn the *shape* of a movement
Details	200 trees, balanced class weights	Conv1d 34→32→64→64 (kernels 5, 5, 3), ReLU, max-pool, global average pool, dropout 0.3, linear → 7 classes
Training	n/a	Adam (lr 1e-3, weight decay 1e-4), 20 epochs, batch 64, **class-weighted loss** so rare classes matter

Normalisation lives inside the CNN (saved mean/std buffers), so the exported file is self-contained and the browser can't apply the wrong statistics.

Step 7: evaluation

Written by `ml/src/train.py` to `ml/reports/` : - **Accuracy:** share of windows labelled correctly. - **Macro-F1:** F1 per class, averaged with equal weight. It's fairer than accuracy when classes are unbalanced: a model can't score well by ignoring rare classes. - **Confusion matrix:** which exercises get mixed up. - **Inference time** per window on the CPU.

Step 8: export to the browser

`ml/src/export.py` : - **Exports the CNN to ONNX**, a portable model format, with softmax added so it outputs probabilities. - **Checks the ONNX output against PyTorch** on real windows: the largest difference must be < 0.001. - **Tries 8-bit quantisation** (smaller file) and keeps it only if predictions change by less than 0.05. - **Writes `exercise_classifier.json` :** labels, feature version, window size, **provenance** (`synthetic` or real) and the measured macro-F1.

Step 9: running in the browser

- `web/lib/ml/classifier.ts` loads ONNX Runtime Web (self-hosted under `/ort`) **only when Auto is turned on**, since it's about 10 MB.
- `web/lib/ml/rolling-window.ts` keeps the last 30 ticks at 15 Hz, the same loop as `resample()` in Python.
- `web/lib/ml/auto-detect.ts` decides when to actually switch exercise. A classifier flickers, and switching resets the rep counter, so a switch must be earned:
- **confidence ≥ 0.7**, for **3 predictions in a row**;
- **Idle never switches**, so resting between sets doesn't lose your exercise.

Keeping Python and TypeScript identical

A model only works on inputs built exactly like its training data. Two safeguards: - **Golden-fixture test:** `python -m src.write_fixtures` writes a clip plus the exact windows Python computes, and a Vitest test checks that `features.ts` produces the same numbers. - **Feature version:** `FEATURE_VERSION` is stored in the model's JSON, so the browser refuses a model built for different features.

1.5 Where the ML stands today (honest status)

- **The shipped classifier is trained on synthetic stick figures.** That's why the app shows a **"Preview"** badge in Auto mode.
- **Its 0.998 macro-F1** (`ml/reports/RESULTS_synthetic.md`) only proves the pipeline runs. **It's not a result and must not be quoted.**
- **Real numbers need real clips** (Phase 2 Step 4): 3+ people, front and side views. Then rerun `prepare → train → export`. The badge disappears once the provenance is real.
- **Rep counting of the original 6** is checked against recorded fixtures (`web/lib/workout/__fixtures__`). Accuracy on real people goes in `ml/reports/rep_counting.md` (to be filled).

1.6 What's next in ML (IMPLEMENTATION_PLAN_2.md)

Step	ML idea
2. Teach a rep	**One-shot calibration:** find the joint that moves most rhythmically in 3 reps and set thresholds from its range. No training.
4. Real clips	Real accuracy numbers, so the Preview badge goes
5. Few-shot model	**Prototypical networks:** learn a 64-number "fingerprint" of a movement, then recognise a *new* exercise from ~5 examples without retraining. Compare against RF, 1D-CNN, GRU, Transformer and **ST-GCN** (a graph network over the skeleton).
6. Rhythm + form ML	Count reps from repetition (autocorrelation) with no thresholds; train a form-quality model and compare it **honestly** with the hand-written rules.

1.7 Glossary

Term	Meaning
Landmark	One body point from the pose model (x, y, z, visibility)
Hysteresis	Two thresholds (enter / exit) so noise near one line can't toggle the state
Window	A fixed-length slice of frames: here 2 s = 30 frames
Feature	A number computed from the data that the model learns from
Augmentation	Making extra training examples by transforming real ones (mirror, rotate, noise)
GroupKFold	Cross-validation where a group (a person) never appears in both train and test
Data leakage	Test information sneaking into training, making scores look better than reality
Macro-F1	F1 averaged over classes equally; robust to class imbalance
1D-CNN	Convolutional network that slides filters along one axis (time)
ONNX	Portable model file format; run in the browser with ONNX Runtime Web
Quantisation	Storing weights as 8-bit integers to shrink the model
Provenance	Where a model's training data came from (synthetic vs real)
MET	Metabolic equivalent: kcal = MET × kg × hours

Part 2: The AI (Gemini) features, briefly

These use a **large language model** we don't train. The design rule: **the AI understands, the formulas calculate.**

Feature	AI does	Numbers come from
Log a meal by text / photo	Names the foods and estimates portions	**USDA food list** (`web/lib/nutrition/foods.ts` , 98 foods). AI estimates are used only when a food isn't in the list, **labelled**, and must pass a calorie check (protein/carbs/fat must roughly add up).
Diet plan	Picks foods and grams	Totals recomputed from the food list and compared with your targets
Calorie targets	**Nothing**	Mifflin–St Jeor BMR × activity, plus a goal adjustment (`targets.ts`)
Coach chat	Writes the reply	Gets a **numbers-only** summary of your training and food (`coach-context.ts`), never video

Safety: - The key is **server-only** (`GEMINI_API_KEY` , used only in `web/app/api/`). - Every AI reply is **validated with zod** before use. - Each user has a **daily quota**, enforced by a database function. - On the free tier the app is limited to 20 requests per day per model, with a fallback model when the main one is busy.

Part 3: Code structure: what each file does

```
repsense/
├── web/                    Next.js app (everything the user sees + the ML that runs live)
│   ├── app/                Pages and API routes (Next.js App Router)
│   ├── components/         React UI, grouped by screen
│   ├── lib/                Pure logic: pose maths, exercises, ML, data, nutrition, AI
│   └── public/models/      Pose model, our ONNX classifier and its metadata
├── ml/                     Python: datasets, training, evaluation, export
├── supabase/migrations/    Database tables + Row Level Security (SQL)
├── docs/                   Report, devlog, design screenshots, this guide
├── .github/workflows/      CI: lint, tests, build, Python tests
├── DESIGN.md               UI design system (source of truth)
├── IMPLEMENTATION_PLAN.md / IMPLEMENTATION_PLAN_2.md   Roadmaps
└── CLAUDE.md               Working rules for the AI assistant
```

> **Rule of thumb:** `lib/` is pure TypeScript (no React), so it can be unit-tested and replayed on recorded clips. `components/` only draws and handles taps.

3.1 `web/app/` : pages and routes

Path	What it is
`layout.tsx`	Root HTML, fonts, theme
`(marketing)/page.tsx`	Landing page (full width)
`(app)/layout.tsx`	Phone-width app shell
`(app)/(tabs)/layout.tsx`	Bottom tab bar layout
`(app)/(tabs)/today/`	Today: suggestion, streak, calories
`(app)/(tabs)/week/`	Week: reps-per-day chart, form trend
`(app)/(tabs)/diet/`	Diet: targets, meals, AI logging, plans
`(app)/(tabs)/coach/`	AI coach chat
`(app)/(tabs)/history/` , `history/[id]/`	Past workouts, one workout's detail
`(app)/(tabs)/profile/`	Account, goal, level, equipment, settings
`(app)/workout/`	The guided workout (picker, then camera, then summary)
`(app)/record/`	Free camera + clip recorder (for training data)
`(app)/clips/`	On-device clip library (play, relabel, export)
`(app)/onboarding/` , `login/`	First-run questions; sign in

Path	What it is
`api/food/parse`, `api/food/photo`	AI meal logging (server only)
`api/diet/plan`, `api/coach`	AI diet plan; streamed coach chat
`auth/callback/`	Supabase sign-in redirect
`manifest.ts`, `robots.ts`, `sitemap.ts`, `opengraph-image.tsx`	PWA install and SEO
`dev/components/`	Preview page for the UI kit

3.2 `web/lib/` : the logic

`lib/pose/` : turning camera output into clean geometry

File	Does
`landmarker.ts`	Loads MediaPipe once per page (GPU, then CPU fallback)
`landmarks.ts`	Names for the 33 landmark indices (`LM.LEFT_KNEE` ...)
`geometry.ts`	Angles, distances, body-line bend, hip offset, torso lean
`filters.ts`	One-Euro smoothing of landmarks
`side.ts`	Picks the camera-facing side (sticky)
`view.ts`	Front vs side view from shoulder width / torso length
`frame-check.ts`	Is enough of the body in frame? Visibility threshold
`draw.ts`	Draws the skeleton and red issue rings on the canvas
`fps.ts`	Measures frames per second

`lib/exercises/` : what an exercise *is*

File	Does
`catalog.ts`	**All 46 exercises as data** (id, name, family, equipment, thresholds, rules)
`families.ts`	The 9 movement families and their defaults (signal, view, cues, MET, pattern)
`signals.ts`	Pose → the one number an exercise counts on; position checks (gates); the chin-over-bar goal
`rules.ts`	Form-rule library (knees caving, hips sag, swing...), resolved by id
`build.ts`	Catalog entry + family defaults → runnable `ExerciseConfig`
`index.ts`	`EXERCISE_CONFIGS` : every built exercise, by id
`state-machine.ts`	Rep counter (hysteresis, rejections) and hold timer
`types.ts`	Shared types and helpers (`jointAngle`, `visible`, `SIDE_LM`, hip and lean rules)
`test-figure.ts`	Test helper: a stick figure built from joint angles

`lib/workout/` : one set, end to end

File	Does
`set-engine.ts`	**The per-frame pipeline:** smooth → side → view → measure → count/time → form → cues. The live camera, clip replay and tests all use it.
`form.ts`	Issue persistence, rep scoring, cue rate-limiting
`replay.ts`	Runs a saved clip through the set engine ("app count")
`summary.ts`	Session totals and per-exercise summaries
`calories.ts`	kcal = MET × weight × time
`plan-store.ts`	Remembers the last picked exercises and sets
`__fixtures__/`	Recorded synthetic clips with known rep counts (regression tests)

`lib/ml/` : the classifier in the browser

File	Does
`features.ts`	TypeScript twin of `features.py` (must match exactly)
`rolling-window.ts`	Last 2 s at 15 Hz, reset on tracking gaps
`classifier.ts`	Loads ONNX Runtime + model lazily; checks the feature version
`auto-detect.ts`	Confidence + streak rules for switching exercise
`__fixtures__/`	Golden features from Python for the parity test

`lib/recording/` : clips for training data

File	Does
`format.ts`	Recording JSON format (shared with `ml/`), labels, the recorder
`clip-store.ts`	Saves clips and optional video in **IndexedDB** (on device only)
`replay.ts`	Playback helpers, "Export all" bundles

`lib/data/` : saving and stats

File	Does
`outbox.ts`	Offline-first queue: upload in order; retry, drop or keep
`sync.ts`	Uploads the queue to Supabase when signed in; profile sync
`rows.ts`	App objects ↔ database rows
`queries.ts`	Server-side reads (sessions, profile, diet)
`stats.ts`	Streak, reps per day, form trend, last-trained dates
`suggest.ts`	Today's session: one legs, one push, one pull exercise

File	Does
`checkin.ts`	Pre-workout check-in (sleep, energy, soreness) → fewer or more sets
`diet-client.ts`	Browser calls for meals, plans and the coach

Other `lib/` folders

Path	Does
`equipment.ts`	Equipment list, presets, which exercises your gear unlocks
`onboarding.ts`	First-run answers (goal, level, equipment, body) in localStorage
`nutrition/`	`targets.ts` (Mifflin–St Jeor), `foods.ts` (USDA list), `match.ts` (food search), `units.ts` ("2 rotis" → grams), `items.ts` (review list), `resize-image.ts` (shrinks meal photos), `profile.ts` (Diet setup answers ↔ targets and database rows)
`ai/`	`gemini.ts` (client, fallback model, errors), `schemas.ts` (zod shapes), `prompts.ts`, `resolve.ts` (AI foods → USDA numbers), `plan.ts`, `coach-context.ts` (numbers-only summary), `route.ts` (sign-in + quota for every AI route)
`supabase/`	Browser/server clients, env checks, session middleware, database types
`keypad.ts`, `safe-next.ts`, `site.ts`, `utils.ts`	Small helpers

3.3 `web/components/`: the screens

Folder	Main files
`ui/`	**Shared design-system parts:** button, card, sheet, segment chip, keypad, tab bar, top bar, chat input… Reuse these; don't restyle per screen.
`workout/`	`workout-flow.tsx` (picker → check-in → camera → summary), `exercise-picker.tsx` (family filter list), `exercise-chooser.tsx` (family → exercise chips), `workout-camera.tsx` (camera loop, drawing, voice, recorder, Auto), `rep-display.tsx`, `angle-graph.tsx`, `set-logger.tsx` (confirm reps and weight), `session-summary.tsx`, `checkin-sheet.tsx`, `placement-tip.tsx`, `auto-detect-bar.tsx`, `use-camera.ts`, `video-recorder.ts`, `voice.ts`
`clips/`	Clip library, preview player, relabel fields, save sheet, export, "app count" vs your count
`diet/`	Diet tab, setup, macro summary, add-food sheet (Snap / Describe / Search), review list, plan card
`coach/`	Coach chat
`today/`, `tabs/`	Today, Week, History, session detail, Profile, sign-in card
`onboarding/`	First-run steps, equipment grid and icons
`charts/`	Recharts bar and line charts (lazy-loaded)
`auth/`	Login form, session sync

3.4 `ml/` : Python

File	Does	Run
`src/features.py`	Feature definition and windowing (twin of `features.ts`)	(imported)
`src/prepare.py`	Clips → windows → `data/processed/*.npz`; augmentation helpers	`python -m src.prepare`
`src/split.py`	Person-wise (or clip-wise) GroupKFold	(imported)
`src/train.py`	Trains RF + 1D-CNN, writes report, confusion plots, models	`python -m src.train`
`src/export.py`	CNN → ONNX, parity check, optional quantisation, metadata	`python -m src.export`
`src/extract_landmarks.py`	Videos in `data/raw/` → landmark clips	`python -m src.extract_landmarks`
`src/synthetic.py`	Procedural stick-figure clips (pipeline testing only)	`python -m src.synthetic`, then `prepare --synthetic`
`src/write_fixtures.py`	Golden features + regression clips for the web tests	`python -m src.write_fixtures`
`src/plot_one_euro.py`	Draws `docs/one-euro.png`	`python -m src.plot_one_euro`
`src/build_foods.py`	Builds the USDA food list for the Diet tab	`python -m src.build_foods`
`tests/test_pipeline.py`	pytest checks of the pipeline	`python -m pytest`
`reports/`	`RESULTS_synthetic.md` (**not a result**), `rep_counting.md` (to fill with real counts)	
`README.md`	Recording format and how to collect data	

3.5 Database: `supabase/migrations/`

Migration	Tables
`0001_init.sql`	`profiles`, `workout_sessions`, `exercise_sets`, `checkins`
`0002_nutrition.sql`	`nutrition_profiles`, `meals`, `meal_items`, `diet_plans`, `coach_messages`, `ai_usage` + `consume_ai_quota()`
`0003_exercise_catalog.sql`	Lets `exercise_sets.exercise` be any catalog id

Every table has **Row Level Security**: you can only read and write your own rows.

Part 4: Following one workout through the code

1. **Pick:** `workout-flow.tsx` shows `exercise-picker.tsx`. Exercises come from `equipment.ts`, filtered by your gear; the suggestion comes from `suggest.ts`.
2. **Camera:** `workout-camera.tsx` starts the camera (`use-camera.ts`) and MediaPipe (`landmarker.ts`), and creates a set engine for the exercise (`createSetEngine(EXERCISE_CONFIGS[id])`).
3. **Every frame:** `set-engine.ts` runs:
4. smooth (`filters.ts`), then pick the side (`side.ts`) and view (`view.ts`);
5. **measure** (the exercise's signal);
6. **count** (`state-machine.ts`);
7. **check form** (`rules.ts` + `form.ts`). It returns reps, phase, issues and events. The camera draws them (`draw.ts`) and speaks cues (`voice.ts`).
8. **Auto mode (optional):** each frame also feeds `rolling-window.ts`. Three times a second, `classifier.ts` predicts on the latest 2 s window, and `auto-detect.ts` may switch the exercise.
9. **Set done:** you confirm reps and weight in `set-logger.tsx`. Calories come from `calories.ts`.
10. **Save:** `rows.ts` turns the sets into rows, then `outbox.ts` and `sync.ts` queue and upload them to Supabase when signed in. Offline, they wait in the queue.
11. **See it:** Today, Week and History read it back through `queries.ts` and `stats.ts`.

Commands cheat sheet

```
cd web && npm run dev        # app at http://localhost:3000
cd web && npm run test       # all unit tests (Vitest)
cd web && npm run lint       # ESLint
cd web && npm run test:ai    # real Gemini calls (needs the key; uses free quota)
cd ml && python -m src.prepare && python -m src.train && python -m src.export   # retrain the
classifier
```